Supplement: Features, Diagnostics, Optimization and Validation

ES 335 / ES 678 Machine Learning, Sem I 2026 Sources: the syllabus calendar names these topics (optimization II; regularization and feature design; regression in practice; calibration; validation and experimental design; decision trees II; KNN distance metrics), but **the lecture PDFs in the course folder cover them only briefly or not at all**. These notes are written from standard material. Compare them with what was done in class.

Step-up path

You need. Linear models, the loss curve of gradient descent, and the idea of a held-out set.

1. Design columns so a simple model can express the truth.
2. Compare with a baseline, and read the residuals.
3. Make training work: scaling, momentum, and diagnosing the loss curve.
4. Repair probabilities and prune trees on held-out data.
5. State how uncertain the final score is.

You can stop when you can say what you would check next when a model disappoints.

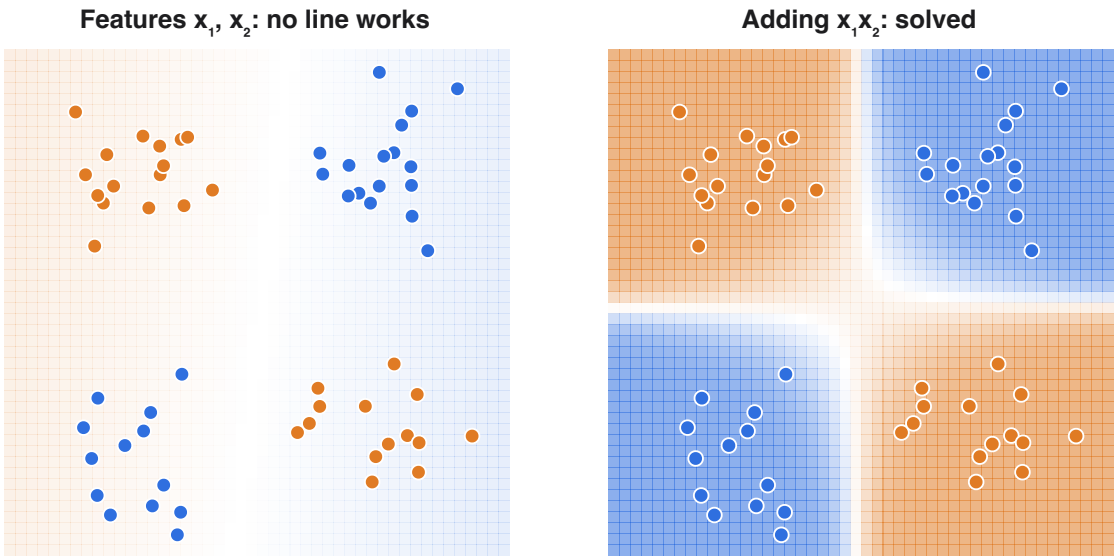
Contents

1. Feature design
2. Regression diagnostics and error analysis
3. Optimization in practice
4. Fixing calibration
5. Cost-complexity pruning
6. Distance metrics
7. Validation designs
8. How uncertain is a test score
9. Auditing a dataset
0. Negative controls and metric mismatch
1. Reproducibility
2. Practice

A1 Feature design

A linear model is linear in its **features**, not necessarily in the raw input. Good features let a simple model express the truth.

- **Polynomials and basis functions.** Replace x by $(x, x^2, \dots)$, or by splines or Fourier terms, and the fit can curve while the optimization stays convex.
- **Interactions.** The term $x_1 x_2$ lets the effect of one feature depend on the other. A linear classifier cannot solve XOR on (x_1, x_2), but can on $(x_1, x_2, x_1 x_2)$.
- **One-hot encoding.** A category with L values becomes L binary columns. With an intercept they are perfectly collinear, because they sum to the intercept column. Drop one, or use regularization.
- **Target encoding** replaces a category by the mean of y in it. Done on all the data, it is leakage, so compute it inside each training fold.
- **Transformations.** Taking logs of skewed positive features often makes relationships more linear.



Same model and data. Shading is the fitted probability. Adding the single column $x_1 x_2$ turns an impossible problem into an easy one.

A2 Regression diagnostics and error analysis

Always compare with a baseline. For regression, predict the training mean. Its mean squared error is the variance of y. For classification, predict the majority class.

Residual plots reveal what a single score hides:

- a curve means the model has the wrong shape;
- a funnel means **heteroscedasticity**, with noise that grows along x, so standard errors are unreliable.

Leverage. An input far from the others has more power over the fit. In simple regression,

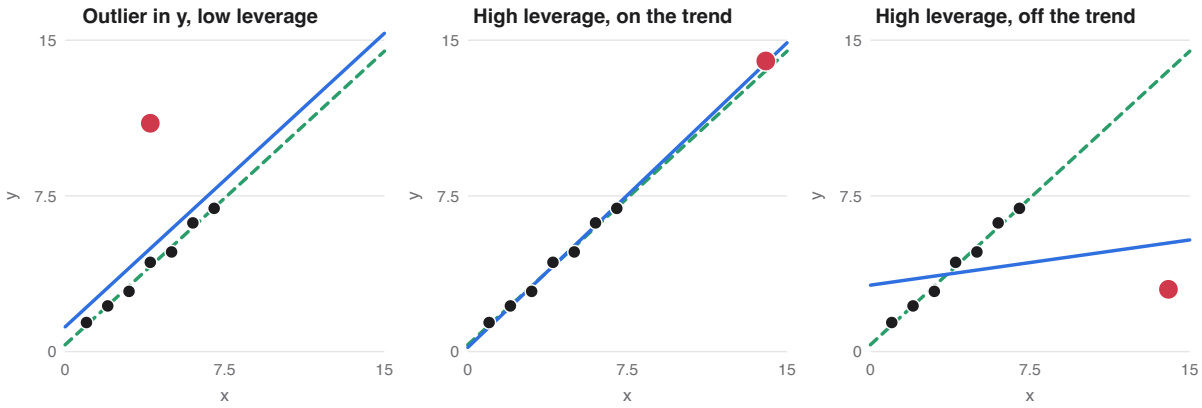
$$h_{ii} = \frac{1}{n} + \frac{(x_i - \bar{x})^2}{\sum_j (x_j - \bar{x})^2}.$$

Cook's distance combines leverage and residual to measure how much the fit moves when point i is removed:

$$D_i = \frac{r_i^2}{p\, s^2} \cdot \frac{h_{ii}}{(1 - h_{ii})^2}.$$

An outlier in y at low leverage may matter little. A point with both a large residual and high leverage can turn the line.

Error analysis. Sort the largest errors, and group them by slice: region, device, month. Aggregate scores hide a slice where the model fails.



The dashed line is the fit without the red point. An outlier near the middle only lifts the line. A far point on the trend changes nothing. A far point off the trend turns the whole line, because it has both leverage and a large residual.

A3 Optimization in practice

Scaling changes conditioning. If one feature is s times larger, the curvature along it grows by s^2, and the largest stable step falls to about $2/s^2$. Standardizing rounds the bowl.

Momentum keeps a running velocity:

$$v_{t+1} = \beta v_t - \eta \nabla L(\theta_t), \qquad \theta_{t+1} = \theta_t + v_{t+1}.$$

Consistent directions build up speed, and oscillating ones cancel, which helps in long valleys.

Schedules. A step that decays over time removes the jitter of SGD. **Adaptive** methods such as Adam scale each coordinate by a running estimate of its gradient size.

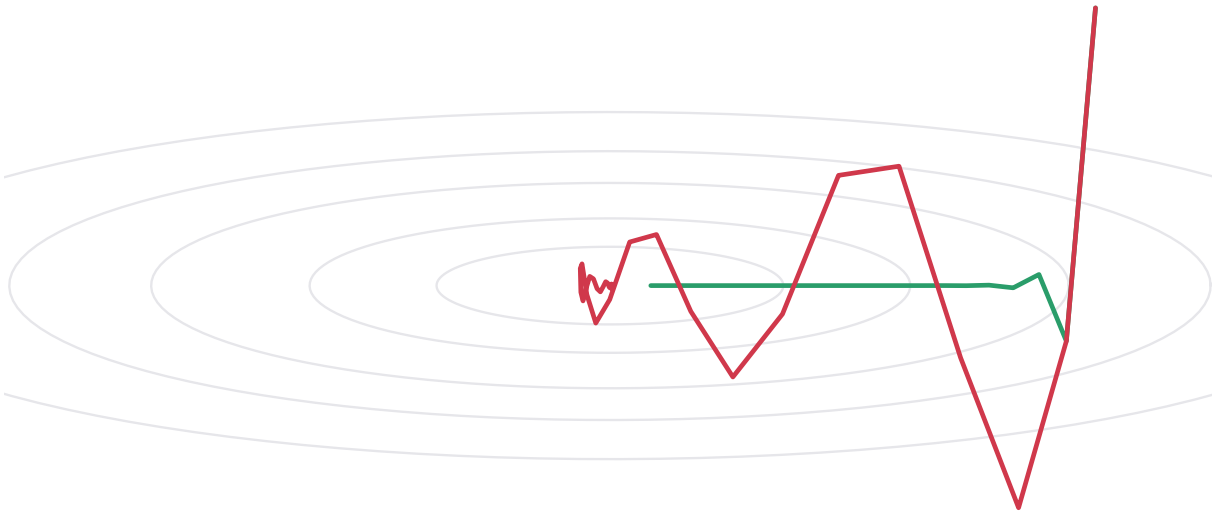
Reading the loss curve.

Curve	Diagnosis
Falls very slowly	Step too small, or badly scaled features
Falls, then flattens	Converging
Zig-zags	Step near the stability limit
Rises or becomes NaN	Step too large, or a bug

Numerical gradient check. Compare your gradient with the central difference, and they should agree to many digits:

$$\frac{f(\theta + h) - f(\theta - h)}{2h}.$$

Plain descent and momentum in a long valley



green: plain descent, red: momentum 0.7

Forty steps from the same start in a valley with condition number 20. Plain descent spends its steps crossing the valley, and momentum cancels that motion and keeps moving along it.

A4 Fixing calibration

Measuring. The **Brier score** is the mean squared error of the probabilities, $\frac{1}{n}\sum(p_i - y_i)^2$. **ECE** is the weighted gap between confidence and accuracy over bins.

Methods, all fitted on a held-out set and never on the training data:

- **Platt scaling.** Fit a logistic regression on the score: $p = \sigma(az + b)$.
- **Temperature scaling.** The one-parameter version: $p = \sigma(z/T)$. The ranking is unchanged.
- **Histogram binning.** Replace each bin's scores by the observed frequency.
- **Isotonic regression.** Fit the best non-decreasing step function. Flexible, but it needs more data.

A5 Cost-complexity pruning

A deep tree overfits. Pruning trades training error against size:

$$R_\alpha(T) = R(T) + \alpha\,|T|,$$

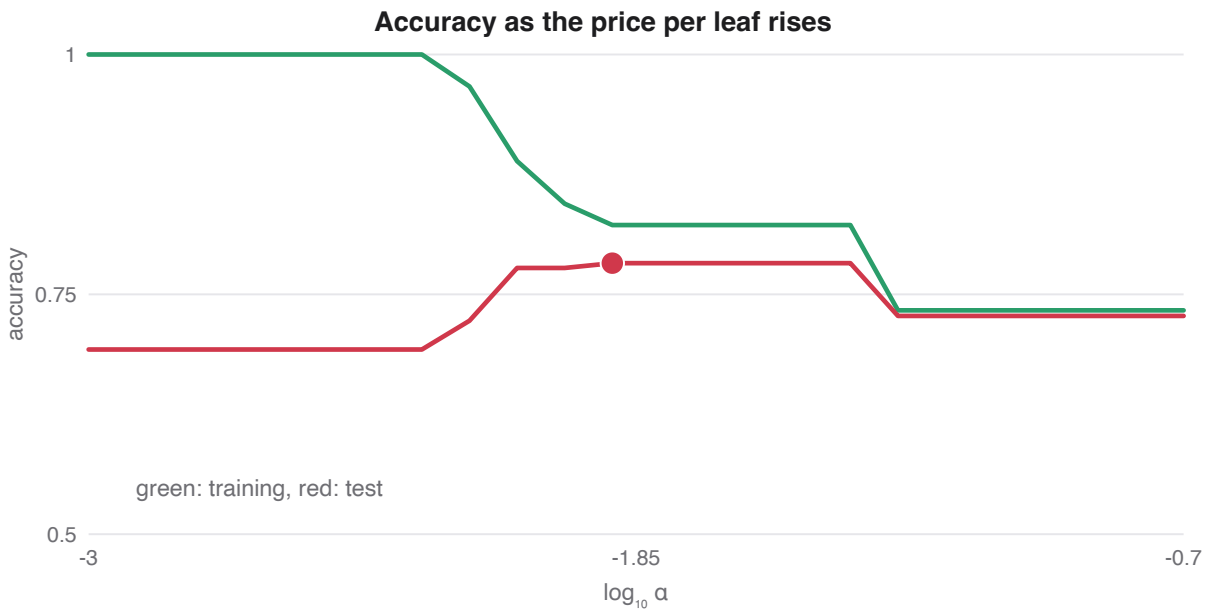
with $R(T)$ the training error and $|T|$ the number of leaves.

For an internal node t, with subtree T_t, collapsing it is worthwhile once

$$\alpha \geq \frac{R(t) - R(T_t)}{|T_t| - 1}.$$

Worked example. A node has error 0.20 if collapsed, and its subtree has error 0.10 over 3 leaves. Then $\alpha = (0.20 - 0.10)/(3 - 1) = 0.05$.

Weakest-link pruning collapses the node with the smallest such α again and again, giving a nested sequence of trees. Pick α by cross-validation.



The green curve is training accuracy, and can only fall as leaves are removed. The red curve is test accuracy, with its best point marked. That peak is what cross-validation looks for.

A6 Distance metrics

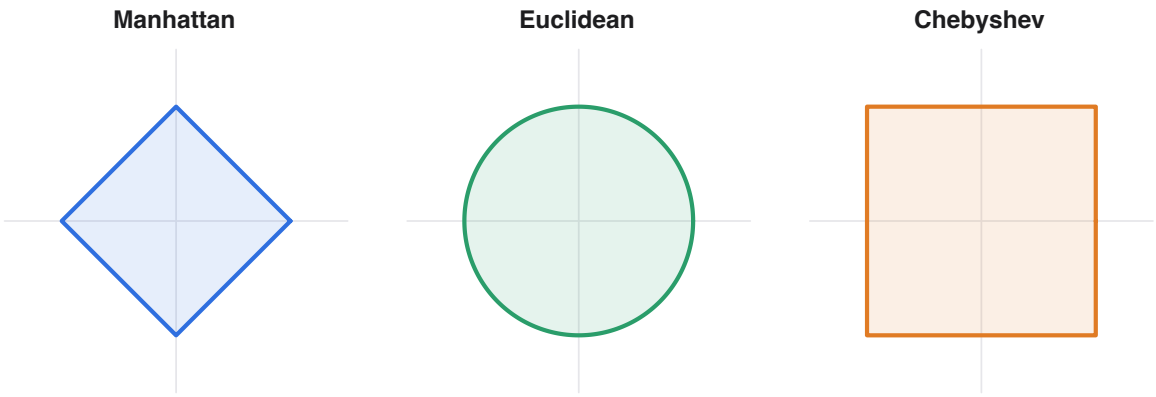
$$d_p(x, z) = \left(\sum_j |x_j - z_j|^p \right)^{1/p}$$

gives Manhattan at $p = 1$, Euclidean at $p = 2$, and the largest coordinate gap as $p \to \infty$. These are the **Minkowski** distances. Their unit balls are a diamond, a circle and a square.

Cosine distance $1 - \cos(x, z)$ ignores length, which suits text and embeddings.

Weighted neighbours vote with weight $1/d$, so closer points count for more.

Scale first, or the widest feature decides who is near.

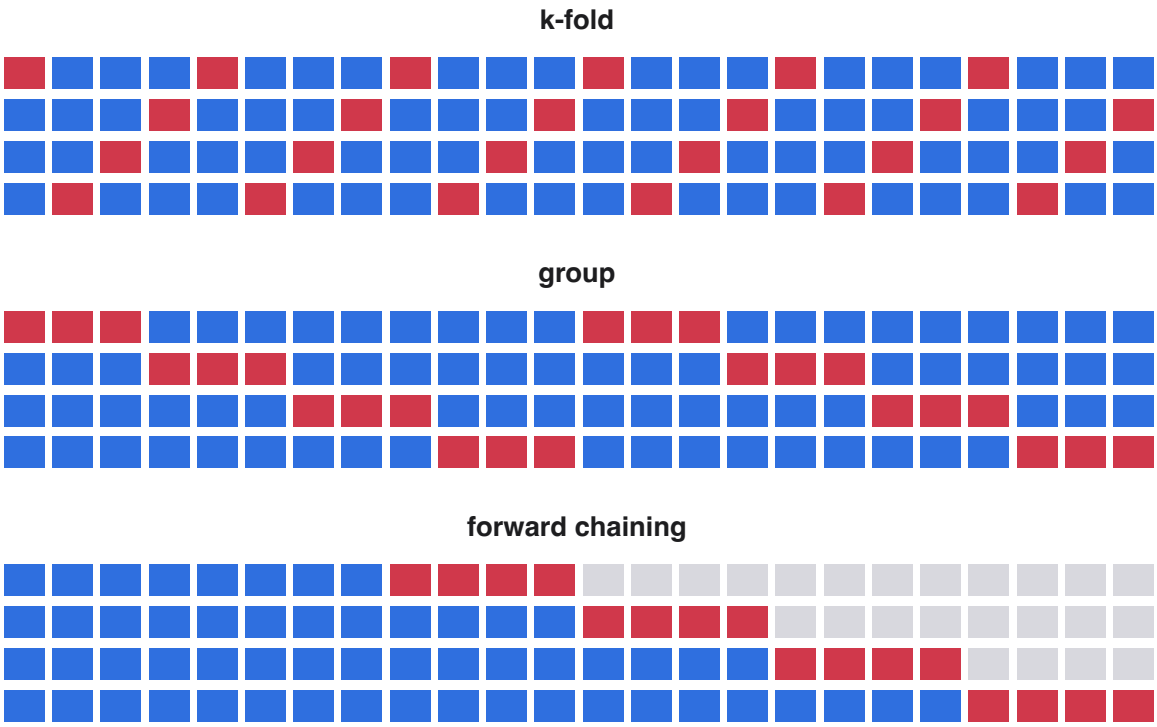


All points at distance one from the centre. Changing the metric changes the shape of "near", so it can change which points are neighbours.

A7 Validation designs

Situation	Use	Why
Independent rows	k-fold	Reuses all the data
Rare classes	Stratified k-fold	Keeps the class mix in every fold
Repeated subjects or sites	Group k-fold	The same subject in train and test is leakage
Time-ordered data	Forward chaining	The future must not train the past
Tuning and reporting together	Nested CV	Selection on a score biases that score

Ablation. Remove one component or feature group, and measure what is lost. That shows what actually does the work.



Rows run left to right and folds top to bottom. Blue trains, red validates, and grey is unused. Group folds keep every three rows together. Forward chaining never trains on the future.

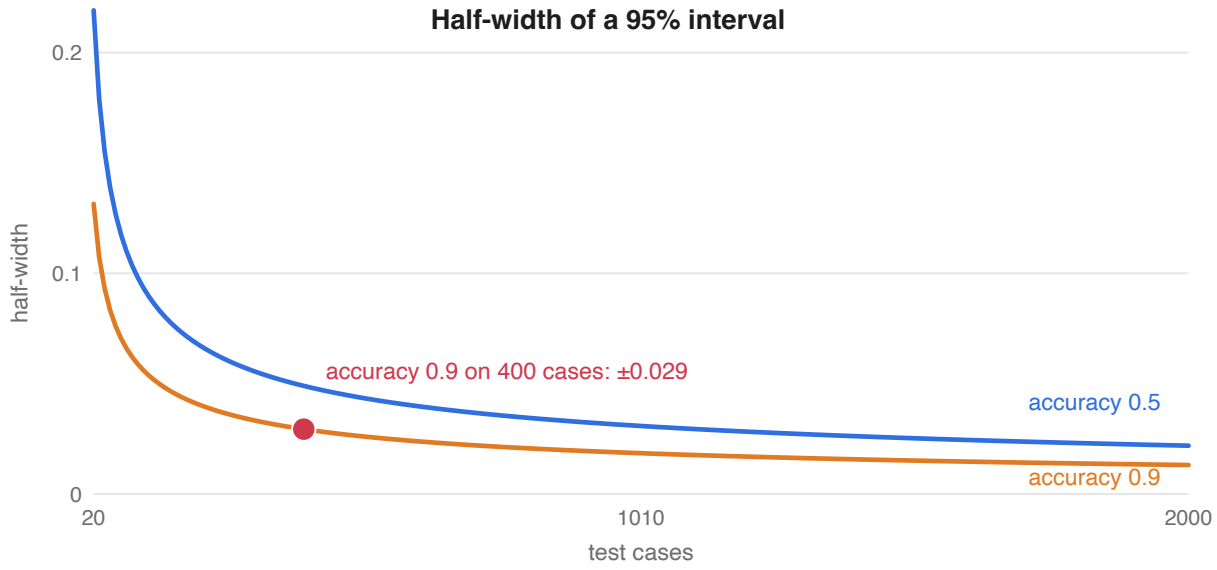
A8 How uncertain is a test score

An accuracy $\hat{p}$ on n test cases has a standard error of about $\sqrt{\hat{p}(1-\hat{p})/n}$, and a 95% interval of about

$$\hat{p} \pm 1.96 \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}.$$

Worked example. $\hat{p} = 0.9$ and $n = 400$ give a standard error of 0.015 and an interval of ± 0.029. A model that is one point better on this test set is not demonstrably better.

To halve the interval, you need four times as many test cases.



The interval narrows with the square root of the test size. The marked point is the worked example, where a one-point difference between models is well inside the noise.

A9 Auditing a dataset

Before fitting anything, go through the table and answer these questions.

Question	What it catches
When was each column recorded, relative to the outcome?	Leakage from the future
Can each value exist? Look at the minimum and maximum.	Impossible values and entry errors
Are rows independent? Look for repeated people, sites or times.	Duplicates that cross the train-test split
What fraction is missing, and why?	Informative missingness
Do units and encodings agree across sources?	Unit and format mismatches
How was the sample collected, and who is absent?	Selection bias

Then **plot** before you summarize. Very different datasets can share the same means, variances and correlation, and a table of summaries will not show it.

Handling missing values. Dropping rows wastes data, and cannot score a new case that has a gap. **Imputation** fills gaps, for instance with the mean or median of the training data. If absence itself is informative, add a binary "was missing" column. Fit every imputer on training data only.

A10 Negative controls and metric mismatch

A negative control is a run that must fail if the pipeline is honest. Shuffle the labels and repeat everything: the test score should fall to chance. If it does not, something is leaking. A **positive control** is a problem with a known easy answer, to show that the pipeline can learn at all.

Metric mismatch is when the number you optimize or report is not the thing you care about: accuracy on rare events, average error when the worst case matters, or a score on a population unlike the one you will serve. Choose the metric from the decision it supports, before looking at results.

A11 Reproducibility

Someone else, or you in a month, should be able to get the same numbers. Record:

- the exact data version and how the splits were made, with their seeds;
- all preprocessing, as code rather than as manual steps;
- the software versions and every hyperparameter;
- which experiments were tried, including those that failed, because reporting only the best run is a form of selection.

Fixing seeds gives repeatability. Reporting results across **several seeds** shows how much is luck.

Practice

P1. A category has five levels, and the model has an intercept. How many one-hot columns can be kept without perfect collinearity?

> Four.

P2. Points $x = 0, 1, 2, 3, 10$. Find the leverage of $x = 10$.

> $\bar{x} = 3.2$. $\sum (x_j - \bar{x})^2 = 10.24 + 4.84 + 1.44 + 0.04 + 46.24 = 62.8$. So $h = 0.2 + 46.24/62.8 \approx 0.936$.

P3. A subtree of 4 leaves has error 0.12, and collapsed it has error 0.18. At what α is it pruned?

$(0.18 - 0.12)/(4 - 1) = 0.02$.

P4. Find the Brier score of predictions $0.8, 0.4$ for labels $1, 0$.

$\frac{1}{2}(0.04 + 0.16) = 0.1$.

P6. A model scores 74% on shuffled labels. What do you conclude?

The pipeline is leaking information about the test labels, or the split is broken. On shuffled labels, an honest test score is at chance.

P5. $\hat{p} = 0.5$ and $n = 100$. Give the half-width of the 95% interval.

$1.96 \times 0.05 = 0.098$.